

UNIT 3: CONTEXT-FREE GRAMMARS AND LANGUAGES

Course Pattern: TE IT - 2024 Pattern (SPPU)	Academic Term: Semester V (2026)
Status: Comprehensive Master Theory Guide	Course Code: PCC-303-ITT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 3.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Chomsky Hierarchy & Regular Grammars	Chomsky Hierarchy classification (Types 0, 1, 2, 3), Left Linear Grammar (LLG), Right Linear Grammar (RLG), Inter-conversions (LLG $\leftrightarrow$ RLG), RG to FA & FA to RG conversions.	High Priority (6-8 Marks Qs)
Context-Free Grammar (CFG) Fundamentals	Formal 4-tuple definition (V, T, P, S), Sentential forms, Yield of derivations, Context-Free Language (CFL) definition, Language generation.	High Priority (6-8 Marks Qs)
Derivations & Parse Trees	Leftmost Derivation (LMD), Rightmost Derivation (RMD), Derivation/Parse Trees, Yield of parse tree, Inherent structural equivalence.	High Priority (8-10 Marks Qs)
Ambiguity in Grammars	Definition of Ambiguous Grammar, Multiple parse trees for single string, Converting ambiguous arithmetic grammars to unambiguous via operator precedence & associativity.	High Priority (8-10 Marks Qs)
CFG Simplification & Normal Forms	Eliminating Useless Symbols, ϵ -productions (Nullable variables), Unit productions ($A \rightarrow B$), Chomsky Normal Form (CNF: $A \rightarrow BC \mid a$), Greibach Normal Form (GNF: $A \rightarrow a\alpha$).	High Priority (8-10 Marks Qs)
CYK Membership Algorithm	Cocke-Younger-Kasami (CYK) Dynamic Programming Membership Algorithm for testing $w \in L(G)$ in $O(n^3 G)$ time using CNF grammar table.	High Priority (8-10 Marks Qs)

Formal Grammars and Chomsky Hierarchy

Definition of a Formal Grammar

A formal grammar is a mathematical system for generating strings of a language via rewriting rules.

DEFINITION:

A formal grammar G is defined as a 4-tuple:

$$G = (V, T, P, S)$$

where:

1. V is a finite set of **Variables** (non-terminals) representing syntactic categories.
2. T is a finite set of **Terminal symbols** disjoint from V ($V \cap T = \emptyset$), forming the alphabet of generated strings.
3. P is a finite set of **Production rules** of the form $\alpha \rightarrow \beta$, where $\alpha, \beta \in (V \cup T)^*$ and α contains at least one non-terminal ($\alpha \neq \epsilon \in T^*$).
4. $S \in V$ is the distinguished **Start symbol**.

The Chomsky Hierarchy of Languages and Grammars

Noam Chomsky classified formal grammars into four hierarchical types based on constraints imposed on production rules $\alpha \rightarrow \beta$:

Type	Grammar / Language	Production Constraint ($\alpha \rightarrow \beta$)	Accepting Automaton
Type 0	Unrestricted / Recursively Enumerable	No restriction ($\alpha \in (V \cup T)^+$ contains ≥ 1 variable)	Turing Machine (TM)
Type 1	Context-Sensitive Grammar (CSG)	$ \alpha \leq \beta $ (non-contracting, except $S \rightarrow \epsilon$)	Linear Bounded Automaton (LBA)
Type 2	Context-Free Grammar (CFG)	$\alpha \in V$ (single non-terminal on LHS)	Pushdown Automaton (PDA)
Type 3	Regular Grammar (RG)	$A \rightarrow aB \mid a$ (Right Linear) or $A \rightarrow Ba \mid a$ (Left Linear)	Finite Automaton (FA)

Regular Grammars: Right Linear and Left Linear Grammars

DEFINITION:

1. **Right Linear Grammar (RLG)**: All production rules are of the form:

$$A \rightarrow wB \quad \text{or} \quad A \rightarrow w \quad \text{where} \quad A, B \in V, w \in T^*$$

2. **Left Linear Grammar (LLG)**: All production rules are of the form:

$$A \rightarrow Bw \quad \text{or} \quad A \rightarrow w \quad \text{where} \quad A, B \in V, w \in T^*$$

Important: A regular grammar must be strictly Right Linear OR strictly Left Linear. Mixing RLG and LLG rules in the same grammar yields a Context-Free Grammar.

Conversion of Regular Grammar (RLG) to Finite Automaton (FA)

Given RLG $G = (V, T, P, S)$:

1. Define FA states $Q = V \cup \{q_f\}$, where q_f is a new final state.
2. Initial state is S , and $F = \{q_f\}$ (if $S \rightarrow \epsilon \in P$, add S to F).
3. For each production $A \rightarrow aB$, add transition $\delta(A, a) = B$.
4. For each production $A \rightarrow a$, add transition $\delta(A, a) = q_f$.

Source: Hopcroft et al., Ch 5, pp. 189–195 | Source: KLP Mishra, Ch 4, pp. 121–130

Context-Free Grammars (CFG) and Derivations

Definition of Context-Free Grammar and Sentential Forms

DEFINITION:

A **Context-Free Grammar (CFG)** is a 4-tuple $G = (V, T, P, S)$ where every production rule in P has the form:

$$A \rightarrow \alpha \quad \text{where } A \in V \text{ and } \alpha \in (V \cup T)^*$$

- **Derivation Step ($\Rightarrow$):** If $A \rightarrow \alpha \in P$, and $\gamma A \delta \in (V \cup T)^*$, then $\gamma A \delta \Rightarrow \gamma \alpha \delta$.
- **Reflexive Transitive Closure ($\Rightarrow^*$):** Sequence of zero or more derivation steps.
- **Sentential Form:** Any string $\alpha \in (V \cup T)^*$ such that $S \Rightarrow^* \alpha$.
- **Sentence:** A sentential form containing only terminal symbols ($w \in T^*$).
- **Context-Free Language (CFL):** $L(G) = \{w \in T^* : S \Rightarrow^* w\}$.

Leftmost and Rightmost Derivations

1. **Leftmost Derivation (LMD):** At each derivation step, the **leftmost non-terminal** in the sentential form is replaced (denoted $\Rightarrow_{lm}$).
2. **Rightmost Derivation (RMD):** At each derivation step, the **rightmost non-terminal** in the sentential form is replaced (denoted $\Rightarrow_{rm}$).

Source: Hopcroft et al., Ch 5, pp. 196–203 | Source: KLP Mishra, Ch 4, pp. 131–138

Parse Trees and Ambiguity in Grammars

Parse Trees (Derivation Trees)

A parse tree is a graphical representation of the hierarchical structure of a derivation in a CFG.

DEFINITION:

In a parse tree for a CFG $G = (V, T, P, S)$:

1. The root node is labeled with the start symbol S .
2. Interior nodes are labeled with non-terminal variables V .
3. Leaf nodes are labeled with terminal symbols T or ϵ .
4. If an interior node labeled A has children $X_1, X_2, \dots, X_k$ from left to right, then $A \rightarrow X_1 X_2 \dots X_k \in P$.
5. The **yield** of a parse tree is the string of leaf terminals obtained by reading leaves from left to right.

Ambiguous Grammars

DEFINITION:

A Context-Free Grammar G is **ambiguous** if there exists at least one string $w \in L(G)$ that has:

- Two or more distinct parse trees, OR
- Two or more distinct Leftmost Derivations, OR
- Two or more distinct Rightmost Derivations.

Eliminating Ambiguity: Operator Precedence and Associativity

Grammars for arithmetic expressions are often ambiguous (e.g. $E \rightarrow E + E \mid E * E \mid \text{id}$). Ambiguity is eliminated by imposing explicit precedence levels:

- Primary expressions / identifiers (F)
- Multiplicative terms ($T \rightarrow T * F \mid F - \text{left-associative } *$)
- Additive expressions ($E \rightarrow E + T \mid T - \text{left-associative } +$)

Source: Hopcroft et al., Ch 5, pp. 205–214 | Source: KLP Mishra, Ch 4, pp. 139–145

Simplification of Context-Free Grammars

Simplification transforms a CFG into a cleaner, equivalent form without altering $L(G)$.

1. Elimination of Useless Symbols

A symbol $X \in (V \cup T)$ is **useful** if there exists a derivation $S \xRightarrow{*} \alpha X \beta \xRightarrow{*} w$ where $w \in T^*$. A symbol is useless if it is non-generating or unreachable.

• **Algorithm:**

1. **Find Generating Symbols:** Set $V_1 = \{A \in V : A \rightarrow w \in P \text{ for } w \in T^*\}$. Iteratively add A if $A \rightarrow \alpha$ where $\alpha \in (V_1 \cup T)^*$. Eliminate non-generating symbols.
2. **Find Reachable Symbols:** Set $Y_1 = \{S\}$. Iteratively add symbols on RHS of productions for variables in Y_1 . Eliminate unreachable symbols.

2. Elimination of Nullable / ε -Productions

An ε -production is of the form $A \rightarrow \varepsilon$. A variable A is **nullable** if $A \xRightarrow{*} \varepsilon$.

• **Algorithm:**

1. Identify all nullable variables ($A \rightarrow \varepsilon \in P$ or $A \rightarrow B_1 \dots B_k$ where all B_i are nullable).
2. For every production $A \rightarrow X_1 X_2 \dots X_m$, substitute all combinations of nullable variables with ε (omitting them), preserving the non-empty options. Delete all original $A \rightarrow \varepsilon$ rules.

3. Elimination of Unit Productions

A **unit production** is of the form $A \rightarrow B$ where $A, B \in V$.

• **Algorithm:**

1. For each variable A , find its **unit pairs** (A, B) such that $A \xRightarrow{*} B$ using only unit productions.
2. For each pair (A, B) , if $B \rightarrow \alpha$ is a non-unit production, add $A \rightarrow \alpha$ to P . Delete all unit productions $A \rightarrow B$.

Source: Hopcroft et al., Ch 7, pp. 231–243 | Source: KLP Mishra, Ch 6, pp. 180–190

Normal Forms: CNF and GNF

Normal forms restrict production rules into standardized canonical forms.

Chomsky Normal Form (CNF)**DEFINITION:**

A Context-Free Grammar G (with $\varepsilon! \in L(G)$) is in **Chomsky Normal Form (CNF)** if every production rule is of the form:

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a \quad \text{where } A, B, C \in V \text{ and } a \in T$$

Conversion Algorithm to CNF:

1. Simplify grammar (eliminate useless symbols, ε -productions, and unit productions).
2. For productions with RHS length ≥ 2 , replace each terminal a with a new variable C_a and add $C_a \rightarrow a$.
3. For productions with RHS length $k \geq 3$ (e.g. $A \rightarrow B_1 B_2 \dots B_k$), introduce cascade variables $D_1, D_2, \dots$ to break the rule into binary pairs:

$$A \rightarrow B_1 D_1, \quad D_1 \rightarrow B_2 D_2, \quad \dots, \quad D_{\{k-2\}} \rightarrow B_{\{k-1\}} B_k$$

Greibach Normal Form (GNF)**DEFINITION:**

A CFG G is in **Greibach Normal Form (GNF)** if every production rule is of the form:

$$A \rightarrow a\alpha \quad \text{where } A \in V, a \in T, \text{ and } \alpha \in V^*$$

Source: Hopcroft et al., Ch 7, pp. 244–248 | Source: KLP Mishra, Ch 6, pp. 191–198

Dynamic Programming Membership: CYK Algorithm

The Cocke-Younger-Kasami (CYK) algorithm determines whether a string w of length n belongs to $L(G)$ for a grammar G in CNF.

DEFINITION:

CYK Triangular Table Construction: Let $w = a_1 a_2 \dots a_n$. Construct a triangular table $V_{\{i,j\}}$ ($1 \leq i \leq n, 1 \leq j \leq n - i + 1$), where $V_{\{i,j\}}$ contains the set of variables $A \in V$ that derive substring $w[i \dots i + j - 1]$ of length j starting at index i .

Algorithm Steps:

1. **Base Level ($j = 1$):** For $i = 1$ to n :

$$V_{\{i,1\}} = \{A \in V : A \rightarrow a_i \in P\}$$

2. **Inductive Substrings ($j = 2$ to n):** For length j , start index i :

$$V_{\{i,j\}} = \bigcup_{k=1}^{j-1} \{A \in V : A \rightarrow BC \in P, \text{ with } B \in V_{\{i,k\}} \text{ and } C \in V_{\{i+k,j-k\}}\}$$

3. **Membership Decision:** String $w \in L(G)$ if and only if start symbol $S \in V_{\{1,n\}}$.

Source: Hopcroft et al., Ch 7, pp. 249–252 | Source: KLP Mishra, Ch 6, pp. 199–202